Mushroom Detection, Instance Segmentation, Counting and Mask-Area Measurement on M18K V2

Digital Image and Video Processing - Course Project Report

Team 10 | Final report manuscript | 5 November 2026

Team members and student IDs: [TEAM LEAD: NAME/ID] | [MINH THINH: FULL NAME/ID] | [THIEN KHANG: FULL NAME/ID]

MVP scope: YOLOv8n-seg reproduction on M18K V2, classical Otsu comparison, ground-truth mask verification, projected-area evaluation, and a Colab inference demo. Pending external submission details are identified in Appendix A.

Abstract

This project develops a reproducible workflow for White Button (WB) mushroom detection, instance segmentation, counting and pixel-area measurement using the M18K V2 image dataset. Two YOLOv8n-seg configurations were trained for 10 epochs: a two-class Baby Bella (BB) and WB model, and a WB-only model. The WB-only checkpoint was evaluated on 450 held-out test images containing 70,192 WB annotations and achieved segmentation AP@50 of 0.9906 and AP@50:95 of 0.7974. To measure projected image area, predicted instance masks were matched to COCO ground-truth masks at IoU >= 0.50. Across 68,794 matches, the mean absolute area error was 211.4 pixels squared and the mean relative error was 8.81%. A simple classical baseline uses grayscale conversion, optional Gaussian smoothing, a self-implemented Otsu threshold, morphology and connected-component counting. On the same 10 randomly selected test images, YOLO achieved foreground-union IoU of 0.9201 and count MAE of 7.3, compared with 0.8662 and 121.2 for the classical baseline. The Otsu implementation matched OpenCV exactly on all 10 images. The project includes an executed Colab inference demo that exports per-instance measurements, annotated images and CSV files. Limitations include dataset imbalance, unresolved annotation and split-overlap concerns, a small classical comparison sample, and the lack of calibrated physical area measurement.

Keywords: M18K V2; YOLOv8n-seg; mushroom counting; instance segmentation; Otsu thresholding; mask area; course project.

1 Introduction

1.1 Problem Statement

Cultivated mushroom beds present many closely spaced, partially touching and overlapping objects. A system intended for measurement must identify individual mushrooms rather than merely label an image as containing mushrooms. The input to our minimum viable product (MVP) is one RGB image. Its outputs are per-mushroom bounding boxes, instance masks, confidence scores, detected WB count, and visible image-plane mask area in pixels squared (px^2). A separate diagnostic workflow loads COCO labels to evaluate detection, segmentation and area error.

Although M18K provides depth images, this implementation uses RGB only. The area is an image-plane measurement, not a physical mushroom cap area or a growth/quality prediction. The two-class experiment is included as a reproduction-oriented comparison; the live demo deliberately focuses on the more prevalent WB category.

The study has two evaluation tracks. The primary track measures YOLO detection and instance-segmentation AP on all 450 predefined test images, then assesses mask-area error on individual matched WB instances. A

small secondary track tests classical foreground segmentation against YOLO on the same 10 seeded test images. Keeping these tracks separate prevents foreground-overlap scores from being mistaken for instance-segmentation AP.

The operational target is a transparent minimum viable system: one saved checkpoint, reproducible input selection, visible masks per instance, a count and an area table. A CSV export lets the user inspect or aggregate individual predictions instead of depending only on visual screenshots.

1.2 Motivation and Applications

Instance-level measurements can support crop inventory, visual crop monitoring and future automated harvesting. In dense beds, counting every bright region as one mushroom is inadequate because neighboring caps can merge, while lighting, substrate and occlusion confuse simple color thresholding. This makes dense mushroom imagery a useful setting for comparing classical image processing with a learned instance segmentation model.

The original M18K study [1] provided RGB-D mushroom imagery and benchmarked detection and segmentation algorithms. Our project reuses its general workflow on an expanded V2 release instead of reproducing the original paper's precise dataset, model schedule or numerical evaluation protocol.

We selected the WB-only model for the final demo because almost all annotations in the extracted V2 copy belong to WB. Nevertheless, the BB+WB experiment is retained because its weak BB performance exposes an important limitation: a high score for the dominant class does not imply reliable performance across both mushroom categories.

1.3 Objectives and Contributions

- Explore the extracted V2 dataset and prepare a reproducible WB-only segmentation subset.
- Train and evaluate two-class and WB-only YOLOv8n-seg checkpoints on the predefined held-out test split.
- Implement and validate Otsu thresholding from scratch, then compare a classical pipeline with YOLO on identical images.
- Quantify projected mask-area error on the full 450-image WB test split.
- Deliver a functioning Colab demo for prediction visualization, counting, mask-area measurement and CSV/image export.

2 Literature Review

2.1 Classical Image Processing Approaches

Classical segmentation frequently converts RGB images into grayscale, suppresses pixel-level noise, separates likely foreground by a threshold and then groups connected pixels. Gaussian filtering smooths local intensity variations. Otsu's thresholding [2] chooses a global grayscale threshold by maximizing between-class variance, eliminating the need to set the threshold by hand. Morphological operations suppress isolated foreground noise, while connected-component labeling extracts regions for area and count measurement. These ideas are described in standard digital image-processing references [5].

Their main advantages are interpretability, low computational cost and limited dependency on annotated training data. However, global color or intensity decisions can be unreliable under heterogeneous lighting and complex backgrounds. Connected components also cannot necessarily separate two mushrooms that touch in the thresholded image. The present project tests these known limitations empirically rather than assuming the classical approach must fail.

Otsu thresholding [2] is particularly suitable for an educational baseline because its criterion can be implemented directly from a histogram and verified against a widely used library. Unlike a trained neural network, the global threshold adapts to the image histogram but does not learn the appearance of mushroom caps; changing the smoothing or morphological operations can therefore alter its behavior substantially in dense scenes.

Classical connected-component labeling has an important interpretive limitation: it counts connected foreground regions, not biological objects. When two caps touch, one connected region may represent multiple annotated mushrooms. The present experiment reports both pixel-level union overlap and instance-count error to expose this mismatch rather than conceal it.

2.2 Learning-Based and Recent Approaches

The M18K paper by Zakeri et al. [1] assessed multiple detection and instance-segmentation algorithms for mushroom imagery, including YOLOv8 and Mask R-CNN. The YOLOv8 family provides separate task-specific detection and segmentation model variants; YOLOv8n-seg is the lightweight segmentation configuration used here [4]. Unlike global thresholding, the model produces a separate predicted mask for each recognized instance, allowing explicit counting and per-mushroom measurement.

COCO [3] provides a well-established representation for instance-level image annotations; the present project uses COCO polygon ground truth and the pycocotools rasterization implementation [7] for evaluation. OpenCV [6] supplies the library reference for our self-implemented Otsu threshold and the classical baseline operations.

The literature positions the methods at different levels of representation. Binary thresholding reasons about image intensity, while instance segmentation predicts object-level masks. Consequently, the same ground-truth annotations must be transformed differently: the classical union metric uses the union of all WB masks, whereas individual area error uses one-to-one matched masks. We use the COCO convention [3], [7] to keep those transformations explicit.

2.3 Positioning of This Work

Our emphasis is a compact, demonstrable course project rather than a new network design. We reproduce a plausible YOLOv8n-seg pipeline on the accessible M18K V2 dataset, add a small classical baseline with code-level verification and ablations, and extend the demo with instance-area measurement. All reported comparisons identify whether the metric is object-level AP, foreground-union overlap, or instance matching error; these are not interchangeable.

3 Methodology

3.1 Dataset

The locally extracted and COCO-indexed M18K V2 data contains 4,495 RGB images across the predefined train, validation and test splits. The class distribution is highly imbalanced: 699,731 WB annotations versus 858 BB annotations (99.88% versus 0.12%). Some public descriptions of the V2 release cite 4,652 images; this report refers only to the 4,495 indexed images verified in the working copy. The downloaded data's specific license terms should be checked before public redistribution.

Table 1. Indexed M18K V2 image splits and COCO category counts.

Split	RGB images	WB annotations	BB annotations
Train	3,596	558,859	686
Validation	449	70,680	75
Test	450	70,192	97
Total	4,495	699,731	858

COCO category IDs originally identify BB as 1 and WB as 2, with an unused Mushrooms category (0). For two-class YOLO, the converted indices are BB=0 and WB=1. For WB-only YOLO, only WB masks are retained and remapped to 0. Six degenerate WB training polygons were excluded, leaving 699,725 valid WB polygons across all splits. This preprocessing retains the original image splits.

In the selected 10-image classical experiment, the ground-truth union is produced by rasterizing each WB polygon and taking the Boolean union of all masks. The instance count is the number of WB annotations for the same image. For the full-test area study, each predicted mask is kept separate at original image resolution. The same annotation source is therefore reused for two distinct but related measurements.

3.2 System Overview

RGB image -> YOLOv8n-seg -> per-instance masks + boxes -> count + area in px² -> visual / CSV output

The training/evaluation workflow uses COCO annotations for masks and splits, and the inference demo requires only the RGB image and existing WB checkpoint. For classroom comparison, a separate pipeline is RGB -> grayscale -> optional Gaussian -> Otsu -> optional morphology -> connected components, producing a binary foreground union and approximate object count.

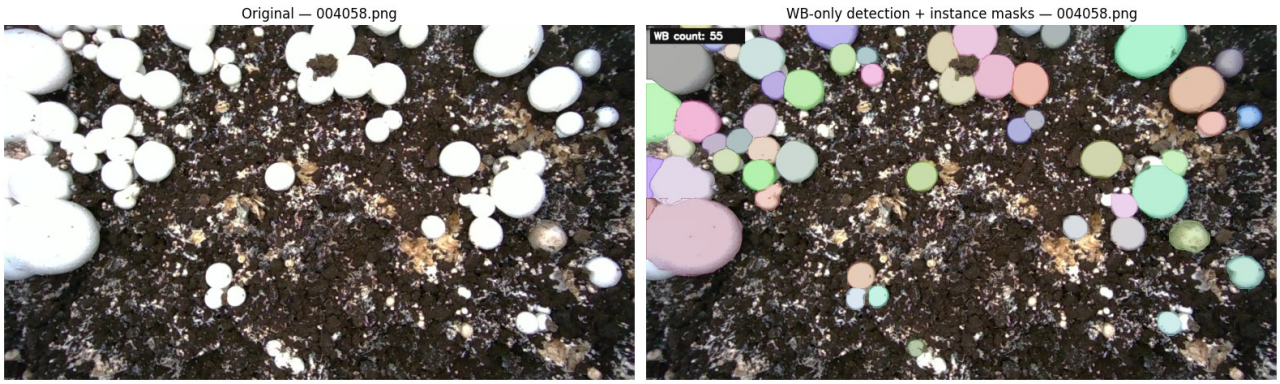


Fig. 1. Executed WB-only inference notebook: RGB input and cleaner predicted instance-mask visualization.

3.3 Classical Baseline: Four Processing Techniques

Technique 1 (smoothing): Convert RGB to 8-bit grayscale and apply a 5x5 Gaussian filter. Technique 2 (intensity segmentation): Compute a global Otsu threshold and classify gray levels above it as foreground. Technique 3 (mathematical morphology): Apply a 3x3 elliptical opening kernel to reduce small positive regions. Technique 4 (region analysis): Apply 8-connected component labeling, retaining components of area at least 100 pixels as approximate objects.

These techniques cover filtering, histogram thresholding, morphology and binary-region analysis. The exact mapping to three course lessons should be confirmed against the lecturer's lecture titles; the course lesson list was not supplied with the rubric.

Table 2. Classical image-processing techniques and their course-topic areas.

Processing technique	Use in this project	Course-topic area (provisional)
RGB-to-grayscale intensity conversion	Prepare 8-bit input for thresholding	Intensity / color representation
5 x 5 Gaussian smoothing	Suppress local intensity variation before Otsu	Spatial-domain filtering
Histogram-based Otsu threshold	Generate global foreground mask	Threshold-based image segmentation
3 x 3 morphological opening	Remove isolated foreground pixels	Binary morphology
8-connected component analysis	Estimate object count; discard areas < 100 px	Binary-region analysis

3.4 Self-Implemented Otsu Threshold

Our function otsu_scratch() builds a 256-bin grayscale histogram, converts it to probabilities, and computes cumulative background weights and means. For each valid threshold t it maximizes between-class variance, selecting the first threshold with the maximum score. The returned binary mask uses pixels greater than the threshold as foreground.

$$sigma_B^2(t) = [mu_T * w_0(t) - mu_0(t)]^2 / [w_0(t) * (1 - w_0(t))]$$

Pseudocode: (1) reject non-uint8 or non-2D inputs; (2) histogram all 256 levels; (3) normalize; (4) compute cumulative weights and means; (5) score candidate thresholds with nonzero class weights; (6) choose argmax; (7) threshold pixels to 0 or 255. We compare the threshold and every pixel against cv2.threshold(..., THRESH_BINARY + THRESH_OTSU), using the identical blurred image. Ten real selected images matched exactly (10/10). This verifies our implementation, not the biological correctness of Otsu segmentation.

Verification protocol: on each of the 10 fixed test images, both implementations receive the same 8-bit grayscale image after identical Gaussian filtering. We compare the numeric threshold and demand exact equality of every 0/255 output pixel. We also test two synthetic grayscale arrays, including a constant-image edge case. This is a functional equivalence test of the algorithm; it is not a claim that Otsu is an accurate stand-alone mushroom instance segmenter.

3.5 Learned Instance Segmentation

Both learned configurations use pretrained YOLOv8n-seg with an image size of 640, batch size of 8, AdamW initial learning rate of 1e-4, seed 42 and 10 completed training epochs. The two-class experiment predicts BB and WB; the single-class experiment predicts WB only. They were trained independently with nonidentical label sets and some differing augmentation defaults. We therefore present their test scores descriptively rather than claiming a controlled effect from removing BB.

The model processes RGB pixels only. We did not retrain models during the Day 4 course demonstration; the notebook loads the saved WB-only checkpoint (SHA-256 beginning 93d993ec6b48 and ending 7961). For qualitative predictions it runs imgsz=640, conf=0.25 and NMS IoU=0.7; this threshold differs from the mAP evaluation sweep, and the two must not be conflated.

For AP, Ultralytics evaluates detections across confidence levels and IoU thresholds; a fixed confidence threshold is not equivalent to the full AP calculation. For the demo and predicted-area measurement, a fixed conf=0.25 provides concrete per-image masks for counting and area export. These two uses of the checkpoint serve different questions and are reported separately.

3.6 Mask Area and Instance Matching

For a binary mask M_i at original image resolution, projected visible area is the number of positive pixels:

$$A_i\ (px^2) = sum\ over\ x,y\ of\ 1[M_i(x,y) > 0].$$

For quantitative area evaluation, COCO polygons are rasterized with pycocotools at their original image sizes. Each predicted instance can match at most one ground-truth instance. Candidate pairs are ranked greedily by mask IoU and retained when IoU >= 0.50; unmatched predictions and ground-truth objects are counted separately. The area mean absolute error and relative error are calculated only for matched pairs:

$$MAE = mean(|A_{pred} - A_{GT}|), \qquad Relative\ error\ (\%) = mean(100 * |A_{pred} - A_{GT}| / A_{GT}).$$

Summing separate instance areas differs from counting pixels in the union because overlapping predicted masks may contribute to the sum more than once. Depth maps or camera calibration were not used; all reported areas are px^2 rather than mm^2.

Because the observed mask boundary is a pixel raster, the area formula counts discrete foreground pixels, not continuous geometric area. Polygon rasterization methods can disagree around boundaries. The implementation was first checked against pycocotools on synthetic and real COCO masks, and the full-test evaluation preserves that rasterization convention when constructing ground truth.

4 Experiments and Results

4.1 Experimental Setup and Metrics

The predefined held-out test split contains 450 images. The main YOLO test evaluation uses Ultralytics box and mask AP@50, AP@50:95, precision and recall. Full-test area evaluation uses 450 WB test images at conf=0.25, NMS IoU=0.7, max_det=1000, retina_masks=True, and mask IoU matching threshold 0.50. The classical comparison uses 10 images chosen by random.Random(42).sample from the test RGB images, with the same 10 COCO labels for every method. That small sample is diagnostic rather than representative of a full-test classical benchmark.

We evaluate foreground-union IoU = intersection / union and Dice = 2*intersection / (predicted foreground + GT foreground). For counting, count MAE is mean absolute difference between predicted and GT instance counts per image. These foreground-union measures do not assess the separation of individual mushrooms, so they are reported separately from instance AP.

For reproducibility, the 10-image sample is generated by sorting the test RGB paths and applying random.Random(42).sample with sample size 10. The same image names and the same COCO WB references are used for every method. Each classical ablation changes only one switch (Gaussian enabled/disabled or morphology enabled/disabled) while retaining Otsu, the area filter, and the connected-component procedure. The YOLO comparison uses its saved WB-only checkpoint; no additional training is performed for this course experiment.

4.2 Reproduction: Two-Class and WB-Only Models

Table 3 reports held-out results from the completed YOLO runs. The class-wise values for the two-class experiment are shown separately because a class-averaged result hides the large WB/BB imbalance.

Table 3. YOLOv8n-seg model results on held-out test images.

Model / evaluated class	Mask AP@50	Mask AP@50:95	Box AP@50
Two-class, overall mean	0.4995	0.3963	0.5066
Two-class, WB only	~0.9890	~0.7870	~0.9920
Two-class, BB only	0.0095	0.0057	0.0213
WB-only model, WB	0.9906	0.7974	0.9921

The two-class model was trained for approximately 1.569 hours; the WB-only model for approximately 1.396 hours on a Tesla T4. The BB test split contains only 97 annotations, and its measured AP is very low. This is compatible with extreme class imbalance but does not independently identify the complete cause. The WB-only test result includes box AP@50:95=0.8399, mask precision=0.9756 and mask recall=0.9631.

Original-paper context: the M18K paper [1], Table 2, reports YOLOv8-N segmentation AP=0.712 at size 640 and AP=0.846 at size 1280. Our dataset release, experiment setup and exact AP implementation have not been harmonized with that study, so a claim that our test score beats its benchmark is not justified.

The original paper used an earlier benchmark data composition and reports its segmentation number under the generic column heading AP; our results are computed by the Ultralytics test implementation and include a class-specific WB-only evaluation on the extracted V2 release. It would be misleading to subtract the two reported AP values and treat the difference as a measured improvement over Zakeri et al. [1]. The defensible result is successful reproduction of the approach and independently documented V2 measurements.

4.3 Baseline Comparison and Ablation Study

Table 4. Classical baseline, ablations and WB-only YOLO on the same 10 test images.

Method (10 images)	Mean union IoU	Mean union Dice	Count MAE	Time, ms/image
Otsu full	0.8662	0.9269	121.2	19.3
Otsu without Gaussian	0.8709	0.9296	122.1	16.5
Otsu without morphology	0.8648	0.9260	122.5	17.4
WB-only YOLO	0.9201	0.9583	7.3	2,797.8

All 10 Otsu thresholds and binary outputs agreed exactly with OpenCV on identical inputs (100% pixel agreement). The full classical method reaches relatively high union IoU but produces major count errors when adjacent mushroom regions connect. YOLO improves both union overlap and instance counting in this small sample. Removing Gaussian blur slightly improves union IoU (0.8709 versus 0.8662) but worsens count MAE (122.1 versus 121.2). Removing morphology reduces union IoU to 0.8648 and slightly worsens count error to 122.5. Therefore, the ablations do not support a simplistic claim that every processing block improves every metric.

The full Otsu-to-YOLO count-MAE gap in the sampled images is 113.9 instances per image (121.2 minus 7.3), whereas the union-IoU difference is 0.0539. This contrast illustrates why connected foreground overlap is an incomplete indicator of object counting quality. The 10-image experiment is too small to estimate the population-wide size of that advantage or to claim statistically significant performance gains.

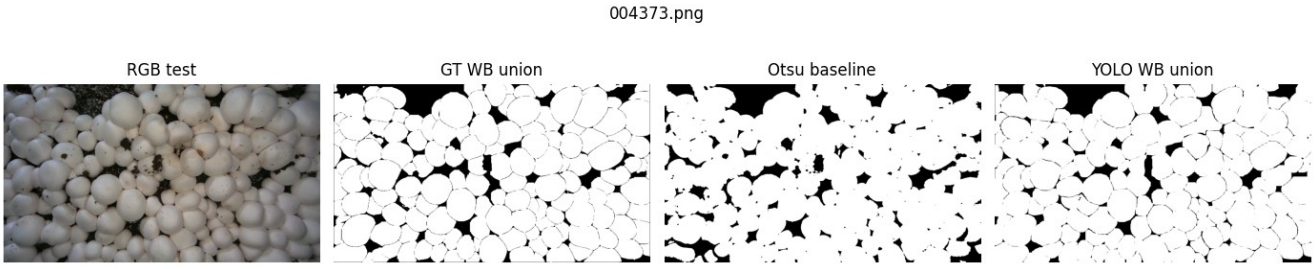


Fig. 2. Same test image: RGB, COCO WB foreground union, classical Otsu foreground, and the union of predicted YOLO instance masks.

4.4 Full-Test Predicted-Area Evaluation

Table 5. Full-test WB instance-mask area evaluation results.

Area metric (450 images)	Result
GT WB annotations / predicted instances	70,192 / 72,182
Matched instances (mask IoU >= 0.50)	68,794
Unmatched predictions / unmatched GT	3,388 / 1,398
GT matching coverage	98.01%
Mean absolute area error	211.4 px^2
Mean / median relative area error	8.81% / 7.31%
Mean matched-mask IoU	0.890
Signed area error mean (pred - GT)	+165.5 px^2
Predicted/GT summed area, matched pairs	1.046

The area errors are computed on the 68,794 one-to-one matched instances, not all 72,182 predictions or all 70,192 ground-truth annotations. On matched pairs, 86.1% of predicted masks are larger than the corresponding ground-truth masks. These data suggest a systematic positive area bias relative to the COCO polygons; they do not identify whether the bias originates from the segmentation model, annotation boundaries, or both.

The 3,388 unmatched predictions and 1,398 unmatched GT annotations are reported but omitted from per-instance area-error averages because a valid one-to-one area reference is unavailable for them under the chosen IoU rule. Coverage is 68,794 / 70,192 = 98.01%, conditional on this particular conf/NMS and mask-IoU matching protocol. A high coverage percentage therefore does not substitute for full mask AP or a manual false-positive inspection.

4.5 Error by Mushroom Size

Table 6. Mask area accuracy by ground-truth mushroom size tercile.

GT area group	GT instances	Matched	Coverage	Mean relative area error
Small (0-33%)	23,378	22,057	94.3%	15.4%
Medium (33-66%)	23,413	23,368	99.8%	7.9%
Large (66-100%)	23,401	23,369	99.9%	3.5%

Most unmatched GT objects are small: 1,321 of the 1,398 unmatched instances lie in the smallest third. Although absolute pixel error is similar across size bands (about 206-221 px^2), small mushrooms have a larger percentage error, and their matched-mask IoU is lower (0.842 for small versus 0.930 for large). These conclusions are conditional on the matching threshold; the most difficult unmatched masks are excluded from the reported area MAE.

The tercile thresholds are approximately 2,093 px^2 and 3,904 px^2 in GT area. The smallest third has lower match coverage (94.35%) than the two larger groups (about 99.8% each), so its 15.4% relative error understates the difficulty of all small GT objects if the unmatched cases are ignored. Large-instance matched-mask IoU averages approximately 0.930, versus 0.842 for small instances.

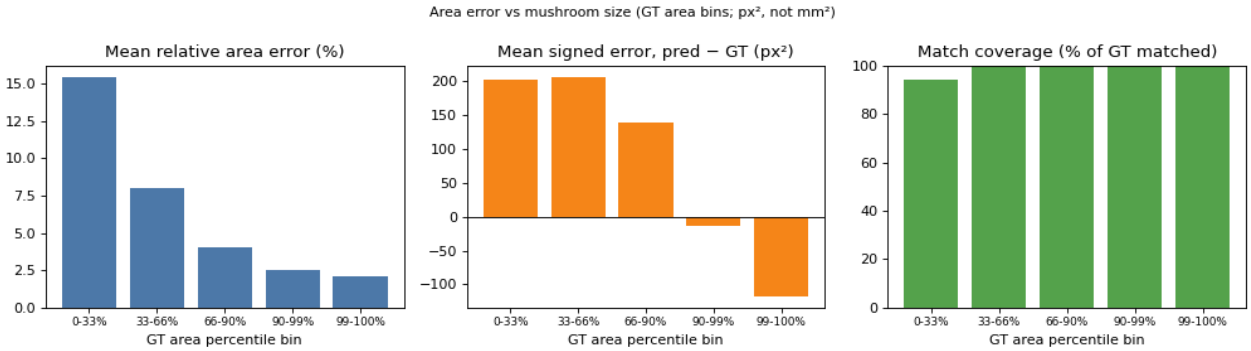


Fig. 3. Full-test area-error trends by ground-truth mushroom size, from the submitted Day 4 evaluation.

4.6 Runtime Performance and Runnable Demo

The executed WB-only Colab demo sampled 10 test images with seed 42 and generated 1,551 predicted instances in total (155.1 per image), with mean predicted instance area of 3,520.11 px². It displayed RGB images alongside colored predicted masks and exported annotated PNG files, mask-ID images, per-instance CSV, per-image CSV and a 14.22 MB results ZIP. These counts are model predictions, not verified GT totals. The notebook completed without reported execution errors.

Runtime observations must be interpreted by hardware and scope. On the Tesla T4 held-out evaluation, Ultralytics reported about 8.3 ms per image for WB-only inference, separately from 1.9 ms preprocessing and 8.9 ms postprocessing. During the 10-image Otsu experiment, YOLO executed on CPU and required an average 2,797.8 ms per image for the notebook-timed prediction call; the classical CPU pipeline took about 19.3 ms per image. These figures are not a fair cross-device speed comparison and have different timing scopes.

Memory reporting: the 10-image notebook measured approximately 1,330.6 MiB process resident set size at its end, which includes the Python runtime and imported libraries. This is not isolated peak memory for a method. The Tesla T4 had about 14.9 GiB reported GPU memory capacity, but a measured YOLO peak GPU allocation was not recorded in the Otsu comparison. A rigorous per-method peak-memory benchmark remains pending.

The CPU timing experiment and T4 validation log are intentionally shown as separate observations. The CPU timing includes the notebook prediction call under its recorded runtime configuration; the GPU log separates preprocessing, network inference and postprocessing. Neither yields a device-independent FPS ranking, and the process RSS number is not a peak GPU memory measurement. A controlled throughput/memory comparison would require equal device, batch, input size and timing boundaries.

5 Discussion

5.1 Error Analysis

The strongest gap between Otsu and YOLO is instance separation. In scenes with many touching white caps, a high percentage of correct foreground pixels can coexist with very inaccurate component counts because several mushrooms merge into one blob. This is visible in Fig. 2, where Otsu produces broad connected foreground and YOLO retains sharper gaps between objects. It explains why foreground-union IoU alone is insufficient to judge a counting system.

For YOLO, small mushrooms are disproportionately difficult in area measurement and matching. Relative error decreases from 15.4% in the smallest third to 3.5% in the largest third. The model also tends to

overestimate ground-truth mask area. The two-class experiment showed particularly poor BB detection, consistent with the very low proportion of BB annotations in the extracted V2 training data.

The area result also has a systematic direction: 86.1% of matched predicted masks exceed the corresponding COCO area, and the sum of matched prediction areas is about 1.046 times the summed GT area. That is evidence of positive raster-area bias relative to annotations, not evidence of physical cap-size overestimation. It may reflect the model, polygon boundary conventions, or both.

5.2 Dataset Integrity and Limitations

During qualitative review, several unusual annotation/image examples were identified, including test images named 004252.png and 004472.png. In at least one example the displayed COCO masks appeared to cover floor or clothing rather than visible mushrooms. Five candidate train/test pairs also shared a perceptual dHash value; the subsequent submitted QC check did not conclusively resolve those original pairs. These observations should be treated as open integrity issues, not as proof of systematic corruption or of data leakage.

Further limitations are the small 10-image classical comparison, the absence of scene-level leakage verification, the short 10-epoch training schedule, and the use of only one WB-only checkpoint in the area evaluation. The WB-only and two-class models were not evaluated under a fully controlled identical-class protocol. Reported image-plane areas are not millimeters squared, and no depth calibration or three-dimensional mushroom-cap surface reconstruction was implemented.

The same WB-only checkpoint was reused in the 450-image test, the area evaluation and the final inference demonstration. This helps provenance, but the downstream pipelines apply different confidence and reporting rules. The 10-image classical comparison estimates sample behavior only; the 450-image area analysis is larger but still conditional on matching and available labels. Neither provides external-field validation at a new farm or production camera.

5.3 Course-Rubric Scope and Future Work

For the minimum viable version, we implemented a classical baseline, a verified algorithm from scratch, a small ablation, a learned model, quantitative tests and a runnable Colab notebook. The deliverables still require a short demonstration video and presentation slides. The course requests a CLI, Gradio or Streamlit demo; a Colab notebook demonstrates the core pipeline but is not itself one of those specified app formats. The exact three-lesson mapping for the listed image-processing techniques also needs confirmation against the actual lecture materials.

Status against the rubric: a verified self-implemented Otsu algorithm, classical alternative, small controlled ablation, labeled quantitative evaluation, and runnable Colab inference notebook are evidenced. A standalone Streamlit/Gradio/CLI application, the short demo video, and a fully isolated peak-memory benchmark are not established by the supplied experiment artifacts. These should be supplied separately if required for full compliance.

A future extension could add a CLI or Gradio front end, repeat the classical comparison on more of the test set, conduct a proper scene-level leakage audit and use verified camera intrinsics with depth data to approximate projected physical area. Such extensions are not claimed as completed.

6 Conclusions

This project delivered a working RGB-based WB mushroom instance-segmentation and measurement workflow using M18K V2. On the 450-image held-out test split, WB-only YOLOv8n-seg obtained mask AP@50 of 0.9906 and AP@50:95 of 0.7974. Full-test area analysis matched 68,794 instances and obtained mean absolute

error of 211.4 px² and mean relative error of 8.81% on those matches. A self-implemented Otsu threshold was verified against OpenCV on 10 test images, and a small classical baseline comparison showed substantially worse counting accuracy than YOLO. The executed inference notebook exports individual predicted masks, counts, image-space areas and figures. The outcome is a successful MVP and reproduction-oriented evaluation, subject to documented annotation, generalization, and course-deliverable limitations.

References

[1] A. Zakeri et al., "M18K: A Multi-Purpose Real-World Dataset for Mushroom Detection, 3D Pose Estimation, and Growth Monitoring," Computers, vol. 14, no. 5, art. 199, 2025, doi: 10.3390/computers14050199.

[2] N. Otsu, "A Threshold Selection Method from Gray-Level Histograms," IEEE Trans. Syst., Man, Cybern., vol. 9, no. 1, pp. 62-66, Jan. 1979, doi: 10.1109/TSMC.1979.4310076.

[3] T.-Y. Lin et al., "Microsoft COCO: Common Objects in Context," in Proc. ECCV, 2014, pp. 740-755. doi: 10.1007/978-3-319-10602-1_48.

[4] Ultralytics, "YOLOv8 Model Documentation," https://docs.ultralytics.com/models/yolov8/ (accessed Nov. 4, 2026).

[5] R. C. Gonzalez and R. E. Woods, Digital Image Processing, 4th ed. Pearson, 2018.

[6] OpenCV, "Image Thresholding / Otsu's Binarization," https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html (accessed Nov. 4, 2026).

[7] COCO Consortium, "pycocotools / COCO API," https://github.com/cocodataset/cocoapi (accessed Nov. 4, 2026).

Appendix A - How to Run the Code

Platform: Google Colab with Python 3.9+; the executed reports used Python 3.13, Ultralytics 8.4.x, PyTorch 2.11.x, OpenCV, NumPy, pandas, matplotlib, pycocotools and gdown. For reproducibility, the exact notebook environment should be frozen in requirements.txt before submission. The full raw M18K dataset and checkpoint are kept outside the Git repository.

Dataset (accessible Google Drive mount): /content/drive/MyDrive/M18K_Reproduction/M18K_V2_data/M18KV2. Test RGB images: test/rgb. COCO test annotations: test/annotations_coco.json. WB-only model: https://drive.google.com/file/d/1Q5iq_-j-RwOTkyiiXyTOc80Cf_xwk7K-/view. The inference notebook downloads it as /content/khang_wb_best.pt if absent.

The experiment notebooks install their Python dependencies in Colab. The inference notebook requires Ultralytics, gdown, pandas, OpenCV and matplotlib; the comparison notebook additionally requires pycocotools and psutil. Use a fresh Colab runtime, mount the authorized Drive, confirm that test/rgb and test/annotations_coco.json exist, and run the notebook cells top to bottom. The trained checkpoint must match the WB-only class map {0: WB}; the recorded SHA-256 is 93d993ec6b48c1fca9d59178afe11377ef20afa3ed0c304841e4d6b36d447961.

- WB inference demo: open M18K_WB_Only_Final_Inference (2).ipynb in Colab, mount the Drive, confirm the dataset path and run all code cells. It selects 10 random test images (seed 42), predicts WB masks, and writes /content/M18K_WB_Final_Results/ and /content/M18K_WB_Final_Results.zip.
- Classical comparison: open M18K_Otsu_Baseline_Ablation (1).ipynb, mount the same Drive and run all cells. It reads matching COCO annotations, checks scratch Otsu against OpenCV, evaluates three classical variants plus WB-only YOLO, and writes /content/M18K_Otsu_Comparison_Results.zip.
- Full area results: retain the submitted Day4_Area_Evaluation folder (area_evaluation.py, area_metrics.json, area_matches.csv.gz, area_per_image.csv, figures). The source checkpoint and dataset must be available to rerun the script; the submitted values already come from an executed test run.

- Main held-out YOLO AP metrics were generated in the original training/evaluation notebooks and stored run folders; the results tables in this report transcribe those recorded outputs.

To reproduce the area evaluation, use the submitted area_evaluation.py with access to the full M18K V2 test split, COCO annotations and the WB-only checkpoint. Its recorded inference settings are imgsz=640, conf=0.25, NMS IoU=0.70, max_det=1000 and retina_masks=True. The match criterion is one-to-one greedy mask IoU >= 0.50. Outputs include area_metrics.json, area_matches.csv.gz, area_per_image.csv and figures; raw dataset files and the checkpoint are intentionally kept outside Git.

Expected demo output: an original RGB image beside a colored prediction-mask image, a per-image predicted WB count, per-instance mask area (px^2), wb_instances.csv, wb_image_summary.csv, and annotated/mask-ID images. Input is an RGB image; no annotation file is necessary for inference.

Repository link: [INSERT FINAL GITHUB REPOSITORY URL]. Course-required short demo video: [TO RECORD / LINK]. Presentation slides: [TO ADD].

Appendix B - Declaration of AI Tool Usage

AI assistance used in preparing the project included ChatGPT (OpenAI) for outlining and revising code/notebooks, debugging suggestions, methodological explanations and drafting/reviewing technical prose. Actual numerical metrics in this document are drawn from the executed experiment notebooks and submitted result files, not AI-estimated values. The team remains responsible for the correctness of code, experiment provenance, citations and the final report.

Any additional AI tools used by individual team members should be listed by name and task before submission: [TEAM TO CONFIRM]. This declaration does not imply that AI generated or verified unseen experimental runs.

AI-assisted edits and explanatory prose were reviewed against the executed notebook logs, supplied metrics JSON/CSV files and the cited original M18K publication. Automated coding assistance is not presented as a substitute for a reproducible run, human review of outlier masks or independent field validation.